

Laboratorio di programmazione per sistemi ciberfisici

18. Esercitazione di riepilogo

Enrico Martini

May 12, 2026

Esercizio 1 (10/02/2023)

Si scriva un programma C che richieda all'utente N valori interi e che li salvi in un array. Il programma deve invocare una funzione `split_prime` che divide l'array sorgente in due array: uno di numeri primi e uno con i restanti valori. Si stampino a video i due array risultato usando un'altra funzione, prevedendo la possibilità che uno di questi sia vuoto.

Esercizio 2 (10/02/2023)

Si implementi la funzione `float mediaCifre(int n)`. Tale funzione deve essere implementata in modo da contenere un algoritmo iterativo che restituisca la media delle cifre che compaiono nel numero intero `n`. Una volta implementata la funzione, è possibile controllarne il corretto funzionamento eseguendo il programma già provvisto di funzione `main`.

Esercizio 3 (10/02/2023)

Si implementi la funzione `int cifraMassima(int n)`. Tale funzione deve essere implementata in modo da contenere un algoritmo iterativo che restituisca la cifra massima che compare nel numero intero `n`, chiesto in input.

Esercizio 4 (10/02/2023)

Si scriva un programma C che richieda all'utente N valori interi e che li salvi nella prima colonna di un array a bidimensionale. Il programma invochi poi una funzione che scriva nella seconda colonna di a, inserendo 1 se il corrispondente elemento nella prima colonna è divisibile per 3 o per 5, altrimenti 0. Si stampino a video i soli valori contenuti nella prima colonna di a per cui si abbia 1 nella seconda colonna, prevedendo la possibilità che questo non si verichi mai.

Esercizio 5 (26/06/2023 B)

Si scriva un programma C che legga da tastiera un array di interi allocato dinamicamente e di dimensione N scelta dall'utente. Il programma deve poi invocare una funzione `change_array()` che sostituisce tutti i valori dispari presenti nell'array con uno 0. Il programma stampi poi il risultato a video.

Esercizio 6 (26/06/2023 A)

Si completi il programma che implementa l'approssimazione in Serie di Taylor del calcolo del seno di un angolo in radianti nell'intervallo (0; 2]:

$$\sin(\alpha) \simeq \sum_{k=0}^n (-1)^k \frac{\alpha^{2k+1}}{(2k+1)!} = (-1)^k \frac{\alpha^{2k+1}}{(2k+1)!} + \sum_{k=0}^{n-1} (-1)^k \frac{\alpha^{2k+1}}{(2k+1)!}$$

Il programma deve chiedere l'inserimento dell'angolo (i.e., α) e verificare che il valore inserito sia nell'intervallo (0; 2]. Deve richiedere il grado di approssimazione per il polinomio di Taylor (i.e., n). Deve calcolare e stampare il seno dell'angolo (6 cifre dopo la virgola) calcolato con polinomio di Taylor. Stampare il valore del seno di tale angolo anche con la funzione `sin()` della libreria `math.h`. Deve calcolare e stampare l'errore di stima (con 6 cifre dopo la virgola) della funzione implementata rispetto alla funzione di libreria. Le funzioni fattoriale e `sintaylor` devono essere implementate come funzioni ricorsive.

Esercizio 7 (26/06/2023 B)

Si implementi l'approssimazione in Serie di Taylor del calcolo del coseno di un angolo in radianti nell'intervallo $(0; 2]$:

$$\cos(\alpha) \simeq \sum_{k=0}^n (-1)^k \frac{\alpha^{2k}}{(2k)!} = (-1)^k \frac{\alpha^{2k}}{(2k)!} + \sum_{k=0}^{n-1} (-1)^k \frac{\alpha^{2k}}{(2k)!}$$

Il programma deve chiedere l'inserimento dell'angolo (i.e., α) e verificare che il valore inserito sia nell'intervallo $(0; 2]$. Deve richiedere il grado di approssimazione per il polinomio di Taylor (i.e., n). Deve calcolare e stampare il coseno dell'angolo (6 cifre dopo la virgola) calcolato con polinomio di Taylor. Stampare il valore del coseno di tale angolo anche con la funzione `cos()` della libreria `math.h`. Deve calcolare e stampare l'errore di stima (con 6 cifre dopo la virgola) della funzione implementata rispetto alla funzione di libreria. Le funzioni fattoriale e `costaylor` devono essere implementate come funzioni ricorsive.

Esercizio 8 (26/06/2023 A)

Si scriva un programma C che legga da tastiera un array di interi allocato dinamicamente e di dimensione N scelta dall'utente. Il programma, lavorando con l'aritmetica dei puntatori, deve poi invocare una funzione `th_array()` che:

- ▶ richiederà all'utente di scegliere una soglia `int th`;
- ▶ creerà un nuovo array, della stessa dimensione del primo e anch'esso allocato dinamicamente, che conterrà 0 se il valore alla cella corrispondente nell'array iniziale è inferiore a `th` e 1 altrimenti
- ▶ restituisce al chiamante il puntatore al nuovo array.

Esercizio 9 (12/07/2023)

Si sviluppi un programma che, dato un array di numeri interi allocato dinamicamente in ingresso, restituisca a video la sua decodifica in stringa, anch'essa allocata dinamicamente. La decodifica avviene in questo modo: se il numero è compreso tra 65 e 90, o è uguale a 32, viene convertito in carattere; viene ignorato altrimenti. Il programma deve lavorare utilizzando l'aritmetica dei puntatori.

Si testi il programma con il seguente array di 25 numeri:

```
1 10 11 72 15 27 69 13 4 76 0 26 76 97 21 98 79 105 3 32 77 48 65 84 55 69
```

Esercizio 10 (14/05/2025)

Si implementi un programma C che legge da tastiera una dimensione N scelta dall'utente e un array di interi allocato dinamicamente, di dimensione N . Si implementi la funzione `minore()` che, lavorando con l'aritmetica dei puntatori:

- ▶ Crea un nuovo array, anch'esso allocato dinamicamente, che contiene solo gli elementi dell'array che sono inferiori alla media dell'array.
- ▶ Stampa l'array.

Esercizio 11

Si sviluppi un programma che, dato un albero binario, conti i nodi interni (che non sono foglie).

Esercizio 12

Si sviluppi un programma che, dato un albero binario, conti i nodi con un solo figlio.

Esercizio 13

Si sviluppi un programma che, dato un albero binario, conti i nodi dal valore pari.

Esercizio 14

Si sviluppi un programma che, dato un albero binario, conti i nodi dal valore maggiore di un certo k , inserito dall'utente.

Esercizio 15

Si sviluppi un programma che, dato un albero binario, verifichi se un valore k è presente, con k inserito dall'utente.

Esercizio 16

Si sviluppi un programma che, dati due alberi binari, verifichi se siano uguali.

Esercizio 17

Si sviluppi un programma che, dato un albero binario, verifichi se l'albero è un albero binario di ricerca (BST).

Esercizio 18

Si sviluppi un programma che, dato un albero binario, trovi la profondità di un nodo con un certo valore k (se esiste).

Esercizio 19

Si sviluppi un programma che, dato un albero binario, conti i nodi al livello k .

Esercizio 20

Si sviluppi un programma che, dato un albero binario, ritorni il massimo grado tra i nodi.

Esercizio 21

Si sviluppi un programma che, dato un albero binario, ritorni il grado medio dei i nodi.

Esercizio 22

Si sviluppi un programma che, dato un albero binario, restituisca il livello massimo con il maggior numero di nodi.

Esercizio 23

Si sviluppi un programma che, dato un albero binario, ne crei una versione specchiata.

Esercizio 24

Si sviluppi un programma che, dato un albero binario, ritorni la somma del percorso a somma massima.

Esercizio 25

Si sviluppi un programma che, dato un albero binario, implementi una funzione che crei una lista dinamica contenente i valori ordinati.

Esercizio 26

Si sviluppi un programma che, dato un grafo non orientato tramite lista di adiacenza, implementi una funzione che restituisce quanti nodi sono pozzi.

Esercizio 27

Si sviluppi un programma che, dato un grafo non orientato tramite lista di adiacenza, implementi una funzione che restituisce quanti nodi hanno grado 1.

Esercizio 28

Si sviluppi un programma che, dato un grafo non orientato tramite lista di adiacenza, implementi una funzione che, dati u e v , ritorna 1 se esiste un arco tra u e v , 0 altrimenti.

Esercizio 29

Si sviluppi un programma che, dato un grafo orientato tramite lista di adiacenza, implementi una funzione che inverte tutti gli archi.